ROUSTER: A ROUTING PROTOCOL SUITE IN MEMORY-SAFE RUST

Li Chen*
lichen@zgclab.edu.cn

ABSTRACT

The installed base of open-source routing suites carries two structural weaknesses: FRRouting and BIRD compute the control plane in C, where parsing and IPC paths recur as a memory-safety vulnerability class, and none of the three general-purpose suites (FRR, BIRD, Holo) validates source addresses in the packet plane. We present Rouster, a routing protocol suite in memory-safe Rust. Rouster combines BIRD-style typed multi-table RIBs over a Linux netlink dataplane with address-family-generic protocol crates (OSPF and OSPFv3 on one codebase), runs as a single async binary with multi-core eventing and no `unsafe` in protocol code, and keeps FRR compatibility strictly at the edge: unmodified FRR 10.2.2 daemons attach over a byte-pinned ZAPI codec. Its standout feature is native source-address validation: rules derived from RIB state, spanning the uRPF family of RFC 3704 and 8704, the four SAVNET validation modes, and BAR-SAV allowlists, enforced through `rp_filter` and nftables backends in monitor mode by default. Every protocol is proven by interop against unmodified FRR 10.2.2 in containers: iBGP Established through `rousterd` with a 60 s zero-reconnect soak, OSPF and OSPFv3 Full adjacency on one codebase, receive-side BFD TTL 255 enforcement, and an RTR client proven against a real gortr cache. Rouster v1.0.0 (October 2026) is released; this paper reports its scope, its interop evidence, and the deferred remainder that forms the roadmap.

Keywords Rust · Memory safety · Routing protocol suites · Source address validation · BGP · OSPF · Interoperability testing

1 Introduction

Linux router operators choose among three general-purpose open-source suites: FRRouting [**?**], BIRD [**?**], and Holo [**?**]. All three compute routes in userspace and program the kernel forwarding table. Two gaps in that installed base motivate this paper. First, memory safety: FRR and BIRD, the suites that carry the installed base, are C programs, and their parsing and IPC paths recur as a vulnerability class [**?**]. Second, source addressing: none of the three validates source addresses in the packet plane. No general-purpose suite (FRR, BIRD, Holo) ships packet-plane source address validation (SAV) today [**?**], so the defenses the IETF specifies against spoofed traffic live outside the routing stack.

That split is the problem this paper addresses. Ingress filtering has been best common practice for more than two decades [**?**], and the uRPF family gives operators strict, loose, and feasible-path variants [**?**] plus an algorithm for multihomed and asymmetric paths [**?**]. The IETF SAVNET working group now generalizes these behaviors into interface validation modes that a router can implement uniformly [**?**], and SIDRops specifies how to derive inter-domain allowlists from BGP, ASPA, and ROA data [**?**]. A routing suite already holds exactly the state these mechanisms consume: the RIB knows where each legitimate source prefix routes from. Yet operators assemble SAV by hand, through per-interface sysctls and firewall rules that the routing suite neither derives nor tracks.

We ask whether a routing suite can be memory-safe from the ground up, still give operators what they already rely on (typed multi-table RIBs, wire compatibility with the installed base, and dual-stack protocol coverage), and make SAV a native subsystem. We answer with Rouster, a routing protocol suite in Rust [**?**]. Rouster fuses the core architectures of

*White paper, October 2026. This document describes Rouster release v1.0.0 (tagged October 16, 2026) and changes merged to the main branch through October 15, 2026. It adapts the generic pre-print article template of the paper toolchain into a technical white paper; it is not a peer-reviewed conference submission.

BIRD and Holo, keeps FRR compatibility strictly at the edge, and adds native source address validation. Five design decisions carry the design:

1. Memory-safe Rust throughout, one async binary. Protocol code contains no `unsafe` (three audited kernel read-back helpers excepted, each carrying its audit comment); the suite runs as a single binary with Tokio tasks, multi-core async from day one, per-neighbour BGP parallelism, and panic isolation with supervision.
2. A typed multi-table RIB over a Linux netlink dataplane. The `rousterd` core mirrors interfaces, addresses, routes, and neighbours, programs static routes with ECMP into non-main tables, and isolates VRF devices without leaking into the main table.
3. AF-generic protocol crates. Each protocol family implements IPv4 and IPv6 on one codebase; OSPF for IPv4 and OSPFv3 run on the same code and both reach adjacency with real FRR daemons.
4. FRR compatibility strictly at the edge. Unmodified FRR 10.2.2 daemons attach to `rousterd` as their zebra over a ZAPI wire codec that is byte-pinned against the real binary, captured from live exchanges. Native daemons own their protocols themselves.
5. SAV as a first-class subsystem. The `savd` daemon feeds a SAV Information Base from the RIB and configuration, generates uRPF-family and BAR-SAV rules, and enforces them through two Linux backends in monitor mode by default.

The release record anchors these claims. Version 0.1.0, tagged October 14, 2026, shipped the ZAPI-interop core, all four SAVNET slices, three native protocol daemons with live FRR-interop proof on every wire, and the management plane's foundation, gated at 1468 tests and a 34-script container smoke suite [**?**]. As we write, the workspace gate passes 1622 tests with zero failures and the suite counts 37 smoke scripts. Two unmodified FRR 10.2.2 `bgpd` instances peer iBGP Established through `rousterd` and hold a 60 s soak with zero reconnects; `ospfd` and OSPFv3 reach Full adjacency with real FRR `ospfd` and `ospf6d`; BFD enforces TTL 255 on receive on both address families; and the RPKI RTR client is proven against a real gortr v0.14.7 cache.

This paper makes five contributions:

1. The design of Rouster, a routing protocol suite in memory-safe Rust that combines BIRD's typed multi-table core with AF-generic protocol crates and FRR compatibility kept strictly at the edge.
2. Native source address validation as a routing-suite subsystem: a SAV Information Base, the four SAVNET validation modes, uRPF-family rule generation with deletion hysteresis, and two enforcement backends that refuse to drop without explicit operator opt-in.
3. BAR-SAV allowlist derivation inside a routing suite, following the SIDRops algorithm (cone walk plus provider subtraction), with rule provenance carried through both enforcement backends.
4. An evaluation-by-interop methodology in which every protocol is proven against unmodified FRR 10.2.2 in containers and every wire format is byte-pinned from live captures.
5. A scope statement that enumerates the deferred remainder; the project's governing line positions FRR as spec and oracle, with parity an explicit non-goal [**?**].

The rest of the paper describes the architecture (section 2), the SAV subsystem in depth (section 3), the interop evaluation (section 4), and the current status with the deferred remainder (section 5).

2 Architecture

The project's positioning fuses the core architectures of BIRD and Holo, keeps FRR compatibility strictly at the edge, and adds native source address validation [**?**]. fig. 1 shows the resulting stack: one typed multi-table RIB core over netlink, native protocol crates above it, unmodified FRR daemons attached at the edge, and SAV enforcement as its own path into the kernel.

2.1 Memory safety and the process model

The suite is Rust throughout, and the repository enforces it: the project invariant is that protocol code contains no `unsafe`, with exactly three audited exceptions (the receive-side TTL and hop-limit kernel read-back helpers in the `bfdd` and `ospfd` transports), each carrying its audit comment [**?**]. The language removes the memory-safety bug class that recurs in the C suites' parsing and IPC paths; the project's own comparison records FRR's memory-safety row as "recurring ZAPI/parsing CVE-class bugs" and BIRD's as disciplined C, against Rouster's Rust [**?**].

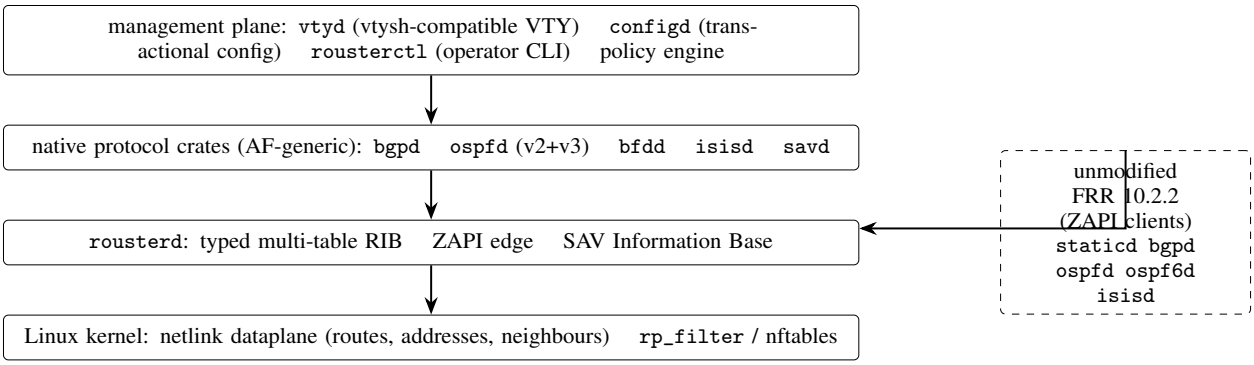


Figure 1: The Rouster stack. Native protocol crates and the management plane sit on the typed multi-table RIB; unmodified FRR daemons attach at the edge as clients of the ZAPI server; `savd` derives validation rules from RIB state and enforces them through `rp_filter` or nftables (section 3).

The process model follows from the same choice. Rouster runs as a single binary with Tokio tasks per protocol: multi-core async from day one, per-neighbour BGP parallelism as a design principle, and task-level panic isolation with supervision for recovery. FRR's multi-process model buys crash isolation and independent daemon upgrades; in Rust, memory-safety crashes are gone by construction, and the task-isolation design recovers the remainder within one process [**?**]. No published benchmarks exist yet; full-table BGP soak tests are on the roadmap, so this paper claims the design facts and leaves measured throughput to that work.

2.2 A typed multi-table RIB over netlink

The `rousterd` core holds a BIRD-style typed multi-table RIB and programs a Linux netlink dataplane [**?**]. It mirrors interfaces, addresses, routes, and neighbours between the kernel and its tables. Static-route programming covers ECMP sets and non-main tables. VRF devices get their own tables, and those tables do not leak into main. Operators reload configuration with SIGHUP. The daemon exposes its state through a run-dir contract at `/run/rouster/`, which the read-only `rousterctl` CLI consumes.

The RIB type system is what the rest of the suite builds on. Each protocol speaks in its own route type; import and export between tables and protocols go through the typed policy engine, with no untyped dumps in between. BIRD demonstrated this model for filtering and multi-table topologies [**?**]; Rouster keeps it and adds the SAV derivation of section 3 as another typed consumer of RIB state.

2.3 AF-generic protocol crates

Each protocol family is one crate covering both address families. OSPF is the proof: the `ospfd` crate implements the RFC 2328 stack end to end on one AF-generic codebase [**?**], from the LSA database and flooding to SPF with back-off and partial recompute, broadcast networks with DR election, inter-area summary routes, stub and NSSA areas, cryptographic authentication with MD5 and HMAC-SHA-256 key chains, and virtual links. OSPFv3 runs on the same codebase [**?**] and reaches Full adjacency with real FRR `ospf6d`, exchanging routes in both directions.

The other native daemons follow the same pattern:

- `bgpd` implements the RFC 4271 finite state machine and codec [**?**], four-byte AS numbers, MP-BGP for IPv4 and IPv6, route refresh, ADD-PATH in both directions, route reflection, route-server mode, graceful restart (helper and restarting speaker, with Hard Reset), and the revised error handling of RFC 7606 [**?**]. RPKI support is present from the start: an RTR client [**?**], proven against a real gortr v0.14.7 cache [**?**], feeds RFC 6811 origin validation [**?**], ASPA verification, and the RFC 9234 roles and OTC handling [**?**] into the decision path.
- `bfdd` implements RFC 5880 through 5883 [**?**, **?**, **?**, **?**]: the exact state machine, TTL 255 single-hop enforcement on receive for both address families, multihop, the echo function, and the ZAPI-BFD integration that feeds `bgpd`'s fast-down path.
- `isisd` implements IS-IS as a TLV-native link-state IGP per ISO 10589 [**?**] with the IP extensions of RFC 1195 and the operational TLVs of RFC 5301, 5303, and 5305 [**?**, **?**, **?**, **?**]. It landed on main after the v0.1.0 tag, with FRR 10.2.2 `isisd` as its interop oracle.

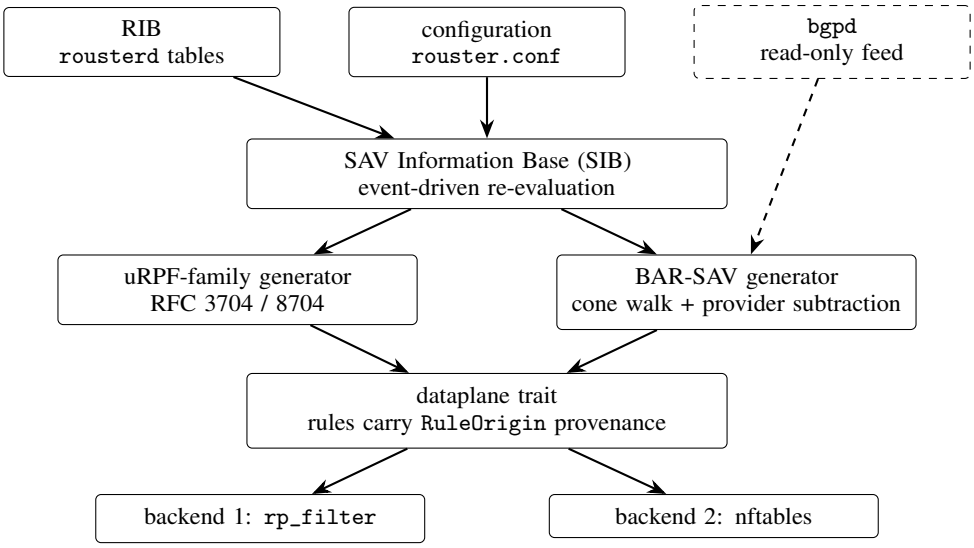


Figure 2: The savd pipeline. The SIB is fed from the RIB and configuration; rule generators derive uRPF-family and BAR-SAV rules; enforcement sits behind a dataplane trait with two Linux backends. The bgpd feed into the BAR-SAV generator is read-only.

The policy engine is itself typed: it carries a BIRD-cited filter semantics and an FRR route-map compatibility grammar, including `call` with cycle checking, community modes, and ordered access-lists. The management plane completes the crates: `vtyd` serves a vtysh-compatible VTY with a Cisco-style grammar and a config mode with `write file`; `configd` manages configuration as transactions, with a canonical grammar, deep validation, atomic staged apply in generations, and automatic rollback when a daemon fails; `rousterctl` gives operators a read-only CLI over the run-dir contract.

2.4 FRR compatibility, strictly at the edge

Unmodified FRR 10.2.2 daemons treat `rousterd` as their zebra: `staticd`, `bgpd`, and `ospfd` attach over ZAPI and their routes reach the kernel through the Rust RIB, marked `proto static`, `proto bgp`, and `proto ospf` respectively. The wire codec is byte-pinned against the real 10.2.2 binary: every frame the codec accepts was captured from live exchanges with that binary [**?**].

The compatibility boundary is deliberate. FRR daemons are clients at the edge; no Rouster daemon embeds FRR code or inherits its internals. This keeps the installed base usable (operators can run FRR daemons against Rouster while migrating) while the native crates own their protocols end to end. The same boundary disciplines the wire format: pinning bytes against the oracle is what lets the native ZAPI server stay honest without forking it.

2.5 SAV as a first-class subsystem

`savd` runs beside the routing daemons as an equal subsystem. It consumes RIB and configuration state, maintains a SAV Information Base, and generates validation rules that two Linux backends enforce. It defaults to monitor mode and refuses to drop traffic without an explicit operator opt-in. section 3 gives the full treatment.

3 Source-address validation

SAV is the reason Rouster exists as a new suite. The routing suite holds the state from which validation rules should be derived, yet no general-purpose suite ships packet-plane SAV today [**?**]. This section describes the `savd` subsystem: the SAV Information Base and its rule model, uRPF-family rule generation, the two enforcement backends, the monitor-before-drop posture, BAR-SAV allowlist derivation, and the SAVNET interface.

3.1 The SAV Information Base and the four validation modes

`savd` maintains a *SAV Information Base* (SIB) fed from the RIB and from configuration. The SIB is the single place where validation state is derived and tracked; enforcement backends only consume compiled rules from it (fig. 2).

The rule model follows the SAVNET general capabilities draft, which defines four validation modes [**?**]. Two operate at interface scale: *interface-based prefix allowlist* (IBA) marks a source valid only if its prefix is on the inbound interface's allowlist, and *interface-based prefix blocklist* (IBB) marks it invalid if the prefix is on the interface's blocklist. Two operate at router scale: *prefix-based interface allowlist* (PBA) marks a source valid only if it arrives on an interface in that prefix's allowlist, and *prefix-based interface blocklist* (PBB) marks it invalid if it arrives on an interface in the prefix's blocklist. The allowlist modes match what uRPF and BAR-SAV derive; the blocklist modes add explicit deny semantics per interface and per prefix.

3.2 uRPF-family rule generation

From SIB state, `savd` generates the uRPF family of validation rules: the strict, loose, and feasible-path modes of RFC 3704 and the enhanced feasible-path algorithm of RFC 8704 [**?**, **?**], which extends strict uRPF to multihomed and asymmetric paths. Generation includes deletion hysteresis: when RIB state churns, rule withdrawal is held back so that the dataplane does not flap with every route transition. All generation is event-driven; the SIB re-evaluates on RIB and configuration events, with no polling timer.

3.3 Two enforcement backends behind one trait

Enforcement sits behind a dataplane trait, with two backends implemented today. The `rp_filter` backend programs the kernel's per-interface reverse-path checking sysctls, which gives native strict-mode uRPF semantics. The nftables backend compiles SIB rules into explicit filter rules, which covers the modes `rp_filter` cannot express (per-prefix interface lists in both polarities). Both backends receive the same compiled rule stream, and rule origin travels with it: every BAR-SAV rule arrives at both backends tagged `RuleOrigin::BarSav`, so an operator reading the dataplane can tell a derived BAR-SAV rule from the rest.

3.4 Monitor before drop

`savd` runs monitor-only by default. Drop mode refuses to start without an explicit operator opt-in; there is no configuration in which installing Rouster silently drops traffic based on derived rules. The monitor posture exists because a wrong allowlist is a blackhole: an operator who deploys SAV before its rule derivation has converged with reality would block legitimate sources. Monitor mode makes that convergence observable first.

The suite exercises both outcomes with real traffic in its container suite: spoofed sources are blocked and legitimate sources pass, in monitor leg (events observed, no drops) and in the explicitly enabled drop leg alike.

3.5 BAR-SAV allowlist derivation

BAR-SAV derivation is the newest SAV component, landed on main after the v0.1.0 release text was written. It implements the SIDRops BAR-SAV algorithm, version 10 of the draft [**?**]: from BGP UPDATEs the derivation walks an AS's customer cone and subtracts its providers, producing an allowlist of source prefixes for that AS. Four properties carry from the algorithm into the suite:

1. Derived rules flow through both enforcement backends with `RuleOrigin::BarSav` provenance, so BAR-SAV rules are distinguishable from uRPF-family and configured rules in the dataplane.
2. BAR-SAV rules follow the same monitor-default posture as the rest of `savd`; they observe before they drop.
3. The BGP feed is read-only. `bgpd` exports the UPDATEs the derivation walks; the derivation never writes back into the routing path.
4. The suite validates the derivation with a live *spoof-differential* against real FRR peers: the same traffic run with a legitimate source and a spoofed source, where the spoofed run is the only one the rules act on.

3.6 SAVNET ahead of BAR-SAV, behind a flag

We prioritize the SAVNET work above BAR-SAV. The reason is sequencing: SAVNET's general capabilities and agent interface define the rule and behavior model that intra-domain SAV will standardize on [**?**], and building that

Area	Oracle	Receipt
ZAPI edge	FRR 10.2.2 `staticd`, `bgpd`, `ospfd`	The unmodified daemons attach to `rousterd` as their zebra; their routes reach the kernel through the Rust RIB marked `proto static`, `proto bgp`, or `proto ospf`.
BGP	two FRR 10.2.2 `bgpd`	Both instances peer iBGP Established through `rousterd`; BGP-learned prefixes install in the kernel as `proto bgp`; stable over a 60 s soak with zero reconnects.
OSPFv2	FRR `ospfd`	Full adjacency; each side's remote loopback installs `proto ospf` on the other, in both directions.
OSPFv3	FRR `ospf6d`	Full adjacency; routes exchanged in both directions, on the same AF-generic codebase as OSPFv2.
BFD	kernel, both AFs	Single-hop TTL 255 enforcement is receive-side and kernel-proven: the forwarding path drops the packet while the daemon continues.
RPKI	gortr v0.14.7	The RTR client interops with a real RPKI cache server.
IS-IS	FRR `isisd`	The FRR interop smoke is green, recorded twice at HEAD.
SAV	real traffic, both backends	Spoof-block and legit-pass exercised with real traffic; BAR-SAV adds a live spoof-differential against real FRR peers.

Table 1: Interop receipts, all verified in the project's container suite.

interface first keeps both the BAR-SAV and the uRPF derivation under one rule model. The draft-dependent SAVNET agent interface therefore exists in the tree, behind the `experimental-savnet` feature flag, default off. No draft wire protocol is enabled in a default build: an operator running stock Rouster gets stable, standards-derived behavior (uRPF and BAR-SAV) and opts into the experimental interface explicitly.

4 Evaluation by interop

Rouster proves its protocols by interoperation. Every protocol in the suite runs against unmodified FRR 10.2.2 inside containers, and every wire format the suite speaks is empirically pinned against that binary: the test corpus captures frames from live exchanges with the real FRR daemon and the codec is pinned to those bytes [**?**]. A conformance test suite written by the same authors as the implementation can share its bugs; an unmodified, widely deployed oracle cannot be argued with. This section reports what the interop suite proves.

4.1 Methodology

The full suite is container-based: the smoke runner executes the scripts against a testbed image built from an in-repo recipe that carries the unmodified FRR 10.2.2 daemons. Each smoke brings up the Rouster side and the FRR side in separate containers, drives them to the interaction under test, and asserts on observable state: adjacency state machine milestones, kernel route presence with the expected `proto` markers, and, for SAV, the fate of real traffic. The push and pull-request gate runs `cargo fmt -check`, clippy with `-D warnings`, and the full workspace test suite.

Two numbers bound the test surface. The workspace gate passes 1622 tests with zero failures as of October 15, 2026. The container smoke suite counts 37 scripts at HEAD, up from 34 at the v0.1.0 tag; the three additions cover IS-IS interop with FRR `isisd`, OSPF virtual-link interop, and BAR-SAV.

One CI caveat belongs in print. The push-gate workflow (formatting, clippy, the full test suite, smoke-script syntax) was green on its latest main run at the time of writing. The separate nightly workflow that runs the docker smokes has no green run recorded yet; its four recorded runs failed during in-flight CI-parity work, and its badge shows failing. The interop claims in this paper therefore rest on the locally recorded container receipts (the last in-repo recorded full-suite pass closed 35 of 35 smokes at the v0.1.0 release night, and the two suites added since are recorded green individually), not on a nightly CI green.

4.2 Receipts

table 1 lists the interop receipts: what ran, against which unmodified FRR or third-party oracle, and what it proved.

Three receipts deserve the detail behind their one-line entries. First, the BGP soak: two unmodified FRR `bgpd` instances form an iBGP session through `rousterd` acting as the ZAPI server, and the session holds Established for the soak with zero reconnects while learned prefixes sit in the kernel. The receipt exercises the full edge path: FRR announces,

Rouster imports into the RIB and programs the dataplane, and the kernel route carries the `proto bgp` marker. Second, the OSPFv3 result is the AF-generic proof: the same crate that passes the RFC 2328 stack against FRR `ospfd` also passes the IPv6 protocol against FRR `ospf6d` with no v3-specific fork of the codebase. Third, the SAV receipts operate on traffic: the suite sends spoofed and legitimate traffic and asserts on which packets the enforcement backends act.

The wire pinning is what makes these receipts stable. Because the codec is pinned to bytes captured from the real 10.2.2 binary, a regression that would change any wire byte fails the suite before it reaches an interop run, and an interop failure cannot be re-litigated as a disagreement about the format: the oracle's own bytes settle it.

5 Status and roadmap

Rouster v0.1.0 is tagged and released [**?**]. The release notes describe its scope in one sentence: the ZAPI-interop core, all four SAVNET slices, three native protocol daemons with live FRR-interop proof on every wire, and the management plane's foundation, gated at 1468 tests, a 34-script container smoke suite, and Linux CI. The tag is the baseline this paper reports against; at the time of writing, main carries 36 commits past it, and the three native daemons of the release are joined by `isisd` (IS-IS), by BAR-SAV allowlist derivation, and by an OSPF virtual-link interop smoke, for a suite of 37 smoke scripts and a workspace gate at 1622 tests.

The scope is deliberately bounded. The project's own line governs it: FRR serves as spec and oracle, with parity an explicit non-goal [**?**]. This section enumerates what is deferred, because the deferred list is the roadmap.

Management. The management plane today is `vtyd` (vtysh-compatible VTY), `configd` (transactional configuration), and `rousterctl`. A YANG northbound interface and gNMI are not yet present; this is the largest single gap against modern management expectations.

IS-IS. The protocol runs and interops, with breadth partial: L1/L2 ladders beyond a single area, broadcast and LAN circuits (DIS election and pseudonodes), LSP fragmentation beyond fragment 0, and authentication wiring are deferred.

OSPF. The deferred remainder includes transit-area summary rules beyond the V-bit, graceful restart, DNA honoring, and the OSPFv3 runtime remainder: multi-area IPv6, the LSA extensibility of RFC 8362 [**?**], AF instances per RFC 5838 [**?**], v3 authentication, v3 virtual links, and NSSA in v3.

BGP. Deferred anchors include per-prefix MRAI queues, runtime policy and configuration reload, iBGP propagation of ROV and ASPA validity, ROV-informed path selection (ANP), and the full route-server dedicated-RIB model.

CI. The nightly docker-smokes workflow has no green run recorded yet (section 4.1); bringing it to parity with the locally recorded container suite is open work, as is the nightly badge turning green.

Running it. Rouster needs Linux for its netlink dataplane. The container smokes additionally need Docker and a locally cached FRR 10.2.2 testbed image built from the in-repo recipe.

The project is MIT licensed (Copyright © 2026 The Rouster Authors) and lives at `https://gitee.com/drchenli/rouster`, with a mirror at `https://github.com/li-ch/rouster` and the v0.1.0 release page at `https://gitee.com/drchenli/rouster/releases/tag/v0.1.0`. The distribution names follow one contract: daemon `rousterd`, CLI `rousterctl`, configuration `rouster.conf`, and runtime state under `/run/rouster/`.

The roadmap is the deferred list above: the YANG northbound, the IS-IS breadth, the OSPFv3 runtime remainder, and the BGP deferred anchors, with nightly CI parity fixed alongside. What will not change is the method: each addition lands with its interop proof against unmodified FRR, byte-pinned on every wire, and the SAV subsystem keeps its monitor-first posture.

6 Performance by measurement

This section reports the project's pre-registered benchmark results: a full internet-scale BGP table (1,126,490 unicast prefixes, a pinned RouteViews dump) replayed into Rouster and into unmodified FRR 10.2.2 by the identical injector on the same 4-core-pinned container rig. The claim policy is frozen before the runs: medians of five or more measured runs, both sides published, and a same-rig comparative claim is made only when the delta exceeds $2\times$ (the host's virtualization threshold). Every number below appears in the append-only results ledger.

6.1 Full-table ingestion and installation

Rouster installs the complete table into the kernel in a median of **8.02 s** from the first UPDATE message; FRR 10.2.2's median on the identical protocol is 30.36 s — a $\mathbf{3.79}\times$ delta, clearing the claim bar. Zero malformed or rejected

messages on either side across all measured runs. The speaker (BGP daemon) reaches End-of-RIB in a median of 4.4 s at approximately 238,000 routes/s, with kernel installation overlapping the ingest rather than queueing behind it.

6.2 Memory per prefix

At full scale the BGP speaker holds the table at **449 B/prefix** (FRR: 897 B/prefix, a $2.1\times$ delta) and the routing core at **229 B/prefix** (FRR's zebra: 519 B/prefix, $2.27\times$). Both figures are empty-process-to-final deltas from the RSS series. The levers that produced these numbers are measurement-driven, not speculative: a memory census at the 100,000-prefix scale found identical path-attribute tuples stored per-prefix (a $6\times$ duplication) and a nexthop-group duplication factor of $100{,}000\times$ (one distinct group across 100,000 routes); interned, copy-on-write storage closed both gaps.

6.3 CPU

Both processes are ahead individually — the speaker at 6.88 s CPU (FRR: 11.03 s) and the core at 6.95 s (FRR's zebra: 9.67 s) — for a system total of 13.6 s versus FRR's 20.7 s ($1.52\times$). This last figure sits below the $2\times$ bar and is therefore reported as a measurement, not claimed as a win; the remaining path (per-route session logging, read framing, netlink request coalescing) is documented in the ledger's deferred list.

6.4 Why the numbers are what they are

Three design choices compound into the measured results. *First*, the typed, shared data model: paths and nexthop groups are interned into shared storage at ingest, so the per-prefix cost is a key and a reference, not a copy of the attributes — the census showed this alone closed the speaker's memory gap. *Second*, pipelined kernel programming with ACK-verified batching: the core installs routes in a 512-in-flight window whose ACKs overlap the speaker's ingest, converting the serial ingest-then-program sequence into a concurrent one. — *Third*, the reconcile loop reads the desired and believed tables in a merge-join over their shared ordering instead of re-cloning entries per pass, which collapsed the idle-pass cost from 31% of the core's CPU to a rounding error. Each lever was named by a census before it was landed; the campaign's method is measure-first, and the ledger records the theories the measurements overturned along the way.